Exploratory Data Analysis and Visualization
Assignment II

Exploratory Data Analysis and Visualization of Credit Card Default Clients: A Case Study with Deep Learning

- **Project Title:** Exploratory Data Analysis and Visualization of Credit Card Default Clients: A Case Study with Deep Learning
- **Course:** U21ADP05 - Exploratory Data Analysis and Visualization
- **Student Name:** RITHIKA C
- **Roll No.:** 24AD092
- **Class :** AD - B
- **Department:** Artificial Intelligence and Data Science
- **Institution:** KPR Institute of Engineering and Technology
- **Year / Semester / Batch:** III Year / V Semester / 2024-2028
- **Course Coordinator:** Ms. Revathi
- **Date of Submission: 15/10/2026**
- **GitHub Repository:** https://github.com/Rithika-Chandrasekaran/credit-card-default-eda-visualization

2. Abstract

Credit card default is a major financial risk for lenders, and early identification of risky clients supports better lending decisions. This study applies exploratory data analysis and visualization to the Default of Credit Card Clients dataset (30,000 clients, 23 features, Taiwan, 2005). Thirty-five duplicate records were removed, undocumented category codes were corrected, outliers were capped using the IQR rule, and four aggregate features were engineered. Ten visualizations following the seven stages of data visualization examined univariate distributions, a Binomial model of late-payment counts, and bivariate and multivariate relationships. A multilayer perceptron with class weighting and tuned hyperparameters was compared with logistic regression and random forest. The MLP achieved a test ROC AUC of 0.776 (95% CI 0.759-0.792) and a recall of 0.648. Repayment status was the dominant predictor, and the MLP performed comparably to random forest.

3. Introduction and Objective

Introduction

Credit cards are a major source of consumer credit, and defaults cause direct financial losses for banks. Lenders therefore need to understand which client characteristics and payment behaviours are associated with default. The case study uses the Default of Credit Card Clients dataset, which records demographic information, credit limits, six months of repayment status, bill amounts and payment amounts for 30,000 clients in Taiwan, with a label showing whether the client defaulted in the following month. The work follows the seven stages of data visualization (acquire, parse, filter, mine, represent, refine, interact) described by Fry (2007) and the exploratory approach of Tukey (1977).

Objectives

- Acquire, clean and preprocess a real-world dataset with more than 15 features.
- Explore univariate, bivariate and multivariate patterns using statistical summaries, tests and at least five visualizations.
- Identify and visualize discrete and continuous distributions.
- Build a deep learning model (MLP) with feature analysis and basic hyperparameter tuning.
- Evaluate it using loss/accuracy curves, a confusion matrix, ROC/AUC and threshold analysis, and derive practical recommendations.

4. Dataset Description

The dataset was obtained from the Kaggle https://www.kaggle.com/datasets/uciml/default-of-credit-card-clients-dataset. It contains **30,000 records and 24 columns** (23 input features and one target), covering April to September 2005 in Taiwan. The target default equals 1 if the client defaulted in the next month. There are **no missing values**, and **22.12%** of clients defaulted, so the classes are imbalanced.

Table 1: Field description

Field(s)	Type	Meaning
LIMIT_BAL	Continuous	Credit limit (NT dollars)
SEX	Categorical	1 = male, 2 = female
EDUCATION	Categorical	1 = graduate school, 2 = university, 3 = high school, 4 = others
MARRIAGE	Categorical	1 = married, 2 = single, 3 = others
AGE	Continuous	Age in years
PAY_1 to PAY_6	Discrete ordinal	Repayment status, Sept down to April: -2 = no consumption, -1 = paid in full, 0 = revolving credit, 1-8 = months of payment delay
BILL_AMT1 to BILL_AMT6	Continuous	Bill statement amount, Sept down to April
PAY_AMT1 to PAY_AMT6	Continuous	Amount paid in previous month, Sept down to April
default	Binary target	1 = default next month

Table 2: Summary statistics of the raw dataset

	count	mean	std	min	25%	50%	75%	max
LIMIT_BAL	30000.0	167484.322667	129747.661567	10000.0	50000.00	140000.0	240000.00	1000000.0
SEX	30000.0	1.603733	0.489129	1.0	1.00	2.0	2.00	2.0
EDUCATION	30000.0	1.853133	0.790349	0.0	1.00	2.0	2.00	6.0
MARRIAGE	30000.0	1.551867	0.521970	0.0	1.00	2.0	2.00	3.0
AGE	30000.0	35.485500	9.217904	21.0	28.00	34.0	41.00	79.0
PAY_1	30000.0	-0.016700	1.123802	-2.0	-1.00	0.0	0.00	8.0
PAY_2	30000.0	-0.133767	1.197186	-2.0	-1.00	0.0	0.00	8.0
PAY_3	30000.0	-0.166200	1.196868	-2.0	-1.00	0.0	0.00	8.0
PAY_4	30000.0	-0.220667	1.169139	-2.0	-1.00	0.0	0.00	8.0
PAY_5	30000.0	-0.266200	1.133187	-2.0	-1.00	0.0	0.00	8.0
PAY_6	30000.0	-0.291100	1.149988	-2.0	-1.00	0.0	0.00	8.0
BILL_AMT1	30000.0	51223.330900	73635.860576	-165580.0	3558.75	22381.5	67091.00	964511.0
BILL_AMT2	30000.0	49179.075167	71173.768783	-69777.0	2984.75	21200.0	64006.25	983931.0
BILL_AMT3	30000.0	47013.154800	69349.387427	-157264.0	2666.25	20088.5	60164.75	1664089.0
BILL_AMT4	30000.0	43262.948967	64332.856134	-170000.0	2326.75	19052.0	54506.00	891586.0
BILL_AMT5	30000.0	40311.400967	60797.155770	-81334.0	1763.00	18104.5	50190.50	927171.0
BILL_AMT6	30000.0	38871.760400	59554.107537	-339603.0	1256.00	17071.0	49198.25	961664.0
PAY_AMT1	30000.0	5663.580500	16563.280354	0.0	1000.00	2100.0	5006.00	873552.0
PAY_AMT2	30000.0	5921.163500	23040.870402	0.0	833.00	2009.0	5000.00	1684259.0
PAY_AMT3	30000.0	5225.681500	17606.961470	0.0	390.00	1800.0	4505.00	896040.0
PAY_AMT4	30000.0	4826.076867	15666.159744	0.0	296.00	1500.0	4013.25	621000.0
PAY_AMT5	30000.0	4799.387633	15278.305679	0.0	252.50	1500.0	4031.50	426529.0
PAY_AMT6	30000.0	5215.502567	17777.465775	0.0	117.75	1500.0	4000.00	528666.0
default	30000.0	0.221200	0.415062	0.0	0.00	0.0	0.00	1.0

5. EDA and Preprocessing

Table 3: Preprocessing summary

Step	Method	Result
Missing values	Checked with isnull(); median imputation coded as a safeguard	None found
Duplicates	drop_duplicates()	35 rows removed (30,000 → 29,965)
Undocumented codes	EDUCATION 0, 5, 6 → 4 (others); MARRIAGE 0 → 3 (others)	Categories consistent with the data dictionary
Outliers	IQR rule (1.5 × IQR), winsorized (capped) for LIMIT_BAL, AGE, BILL_AMT1-6, PAY_AMT1-6	No rows lost
Feature engineering	AVG_BILL, AVG_PAYMENT, UTILIZATION (AVG_BILL / LIMIT_BAL), AVG_DELAY	4 new features
Encoding	One-hot (drop_first) for SEX, EDUCATION, MARRIAGE; PAY_x kept as ordinal numeric	30 input features
Scaling	StandardScaler fitted on the training set only	Prevents data leakage
Splitting	Stratified 70/15/15	Train 20,975 / Validation 4,495 / Test 4,495

Methods and Insights:

The dataset had no missing values, so the imputation step acted only as a safeguard. Thirty-five duplicate rows were removed. Because the dataset has no client identifier, identical rows could in principle belong to different clients, but the number is small (0.12%) and does not affect the conclusions. EDUCATION contained undocumented codes (0, 5, 6) and MARRIAGE contained code 0, all merged into “others”. The monetary variables were strongly right-skewed. Many values flagged by the IQR rule were legitimate high-value customers, so outliers were capped rather than deleted. The six bill-amount columns were highly correlated with each other minimum pairwise correlation, so aggregate features were engineered to summarize them. Standardization was fitted on the training set only, and the split was stratified so each subset keeps the same default rate (about 22%).

6. Data Visualization

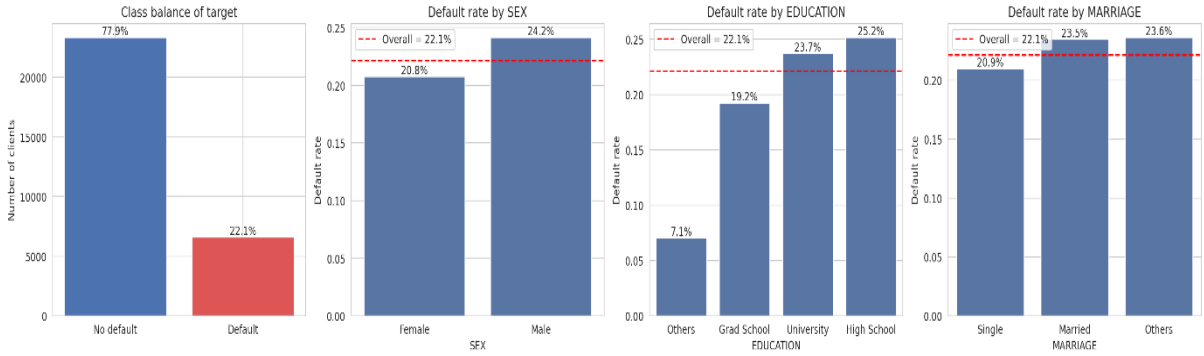
Table 4: Seven stages of visualization

Stage	What was done	Where
1. Acquire	Downloaded from UCI with ucimlrepo	Data loading
2. Parse	Inspected types, category codes, structure	Data understanding
3. Filter	Removed duplicates, fixed codes, capped outliers	Cleaning, Fig. 6
4. Mine	Skewness, chi-square, Mann-Whitney U tests	Statistical tests
5. Represent	Bar, histogram, box, scatter, heatmap, pairplot	Figs. 1-9
6. Refine	Log transform, Q-Q plots, before/after capping, log axes, labels	Figs. 2, 5, 6
7. Interact	Plotly hover/zoom plots (HTML in the repository)	Figs. 10-11

Statistical tests

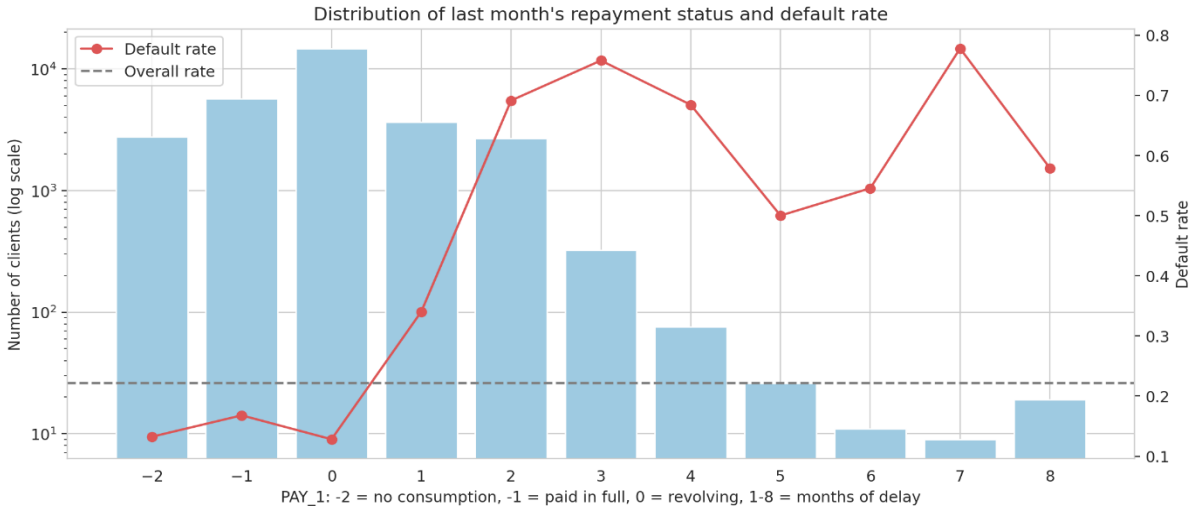
Chi-square tests showed that SEX, EDUCATION, MARRIAGE, PAY_1 had $p < 0.05$, with χ^2 values. PAY_1 had by far the largest χ^2 statistic. Mann-Whitney U tests showed that LIMIT_BAL / AGE / BILL_AMT1 / PAY_AMT1 differ significantly between defaulters and non-defaulters, with the medians. With 29,965 records, even small differences become statistically significant, so effect size matters more than the p-value.

Figure 1: Target balance and default rate by category



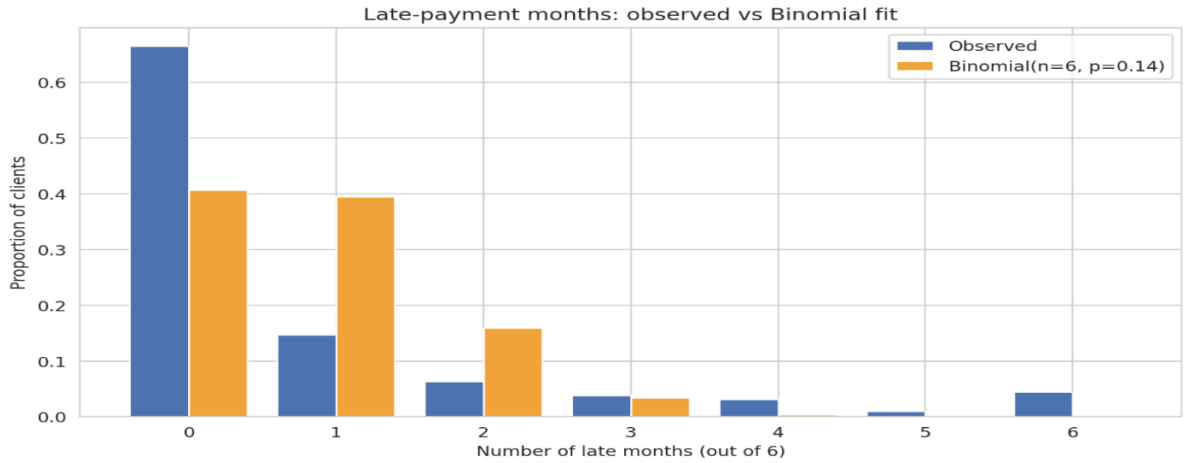
- **What it shows:** the number of defaulters and the default rate by sex, education and marital status, against the overall rate (22.1%).
- **Why created:** to check class imbalance and whether demographics matter.
- **Insight:** about 22% of clients default, so the data are imbalanced. Default rates differ by [confirm: range, e.g. a few percentage points] across demographic groups, so demographic variables are weak predictors compared with payment behaviour.

Figure 2: PAY_1 distribution and default rate



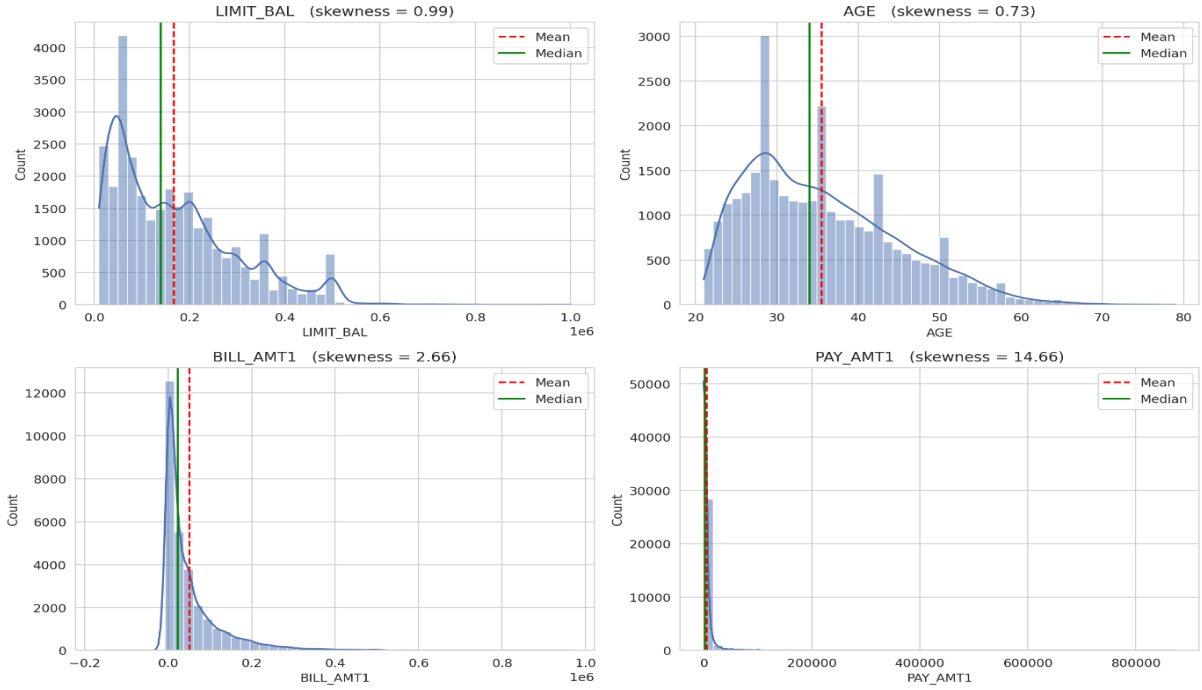
- **What it shows:** the discrete distribution of PAY_1 (bars) with the default rate at each level (line).
- **Why created:** repayment history is the most direct behavioural signal, so it was examined directly.
- **Insight:** most clients are at -2, -1 or 0, and the default rate rises with the length of delay. The default rate at PAY_1 = 2 is [confirm: Cell 27] against [confirm] at PAY_1 = 0.

Figure 3: Binomial fit for late payments



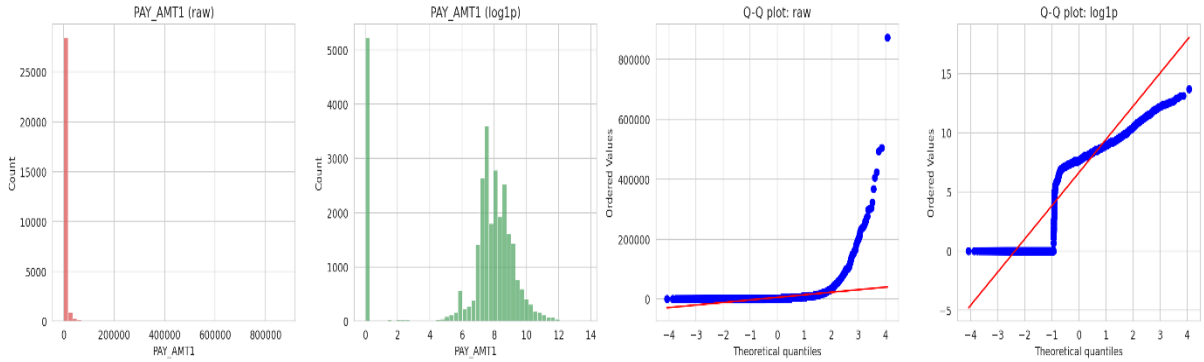
- **What it shows:** the observed distribution of late months out of six compared with a Binomial(n=6, p=[confirm: p_hat]) distribution.
- **Why created:** to examine a discrete probability distribution (CO4).
- **Insight:** the observed variance is [confirm: obs var] against [confirm: binomial var]. If the observed variance is larger, the data are overdispersed, meaning late payments cluster in the same clients and months are not independent. If they are similar, say the Binomial model fits adequately.

Figure 4: Continuous distributions



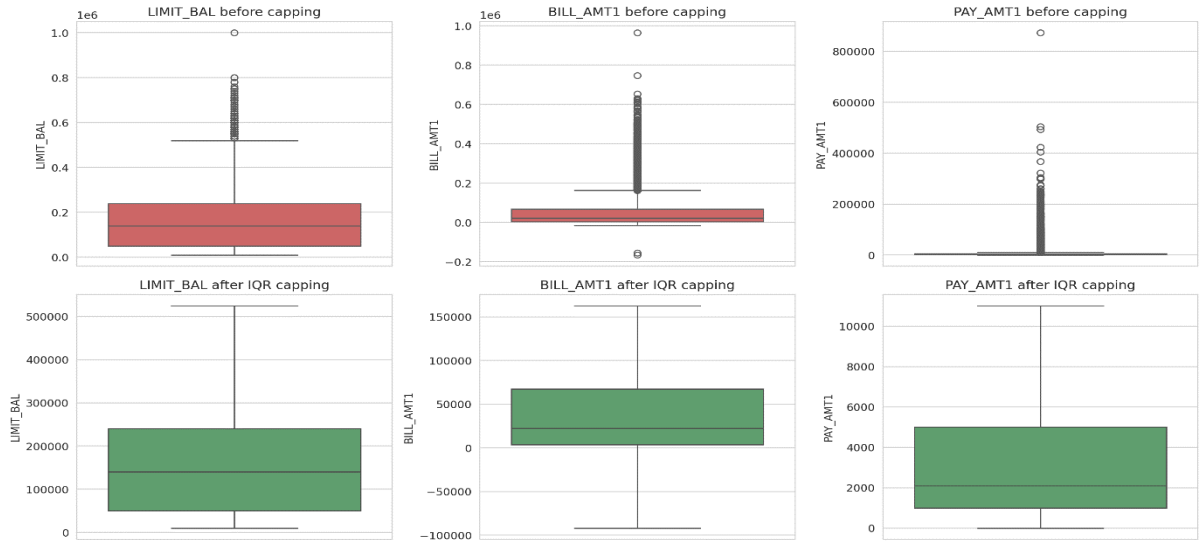
- **What it shows:** shape, centre and spread, with the mean and median marked.
- **Why created:** to choose suitable handling for skewed variables.
- **Insight:** skewness values are [confirm: LIMIT_BAL, AGE, BILL_AMT1, PAY_AMT1 skew]. For right-skewed variables the mean exceeds the median, so the median describes a typical client better. PAY_AMT1 is [confirm: the most skewed], which motivates a transformation.

Figure 5: Log transform and Q-Q plots



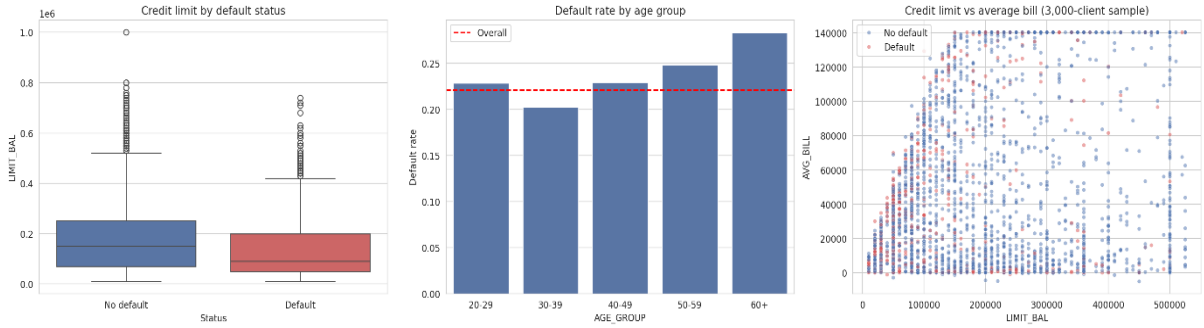
- **What it shows:** the distribution and its normal Q-Q plot before and after log1p.
- **Why created:** the Refine stage, improving the representation of a skewed variable.
- **Insight:** skewness fell from [confirm: raw] to [confirm: log1p]. The Q-Q plot moves closer to the line, but the large number of zero payments still leaves a visible step, so the variable is not truly normal.

Figure 6: Outliers before and after capping



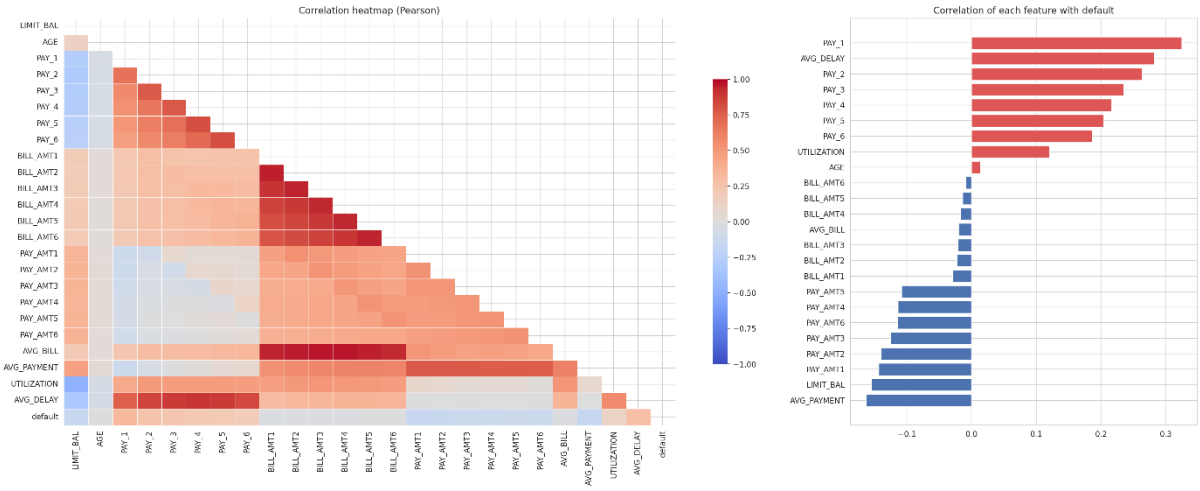
- **What it shows:** the effect of winsorizing on LIMIT_BAL, BILL_AMT1 and PAY_AMT1.
- **Why created:** to justify the cleaning choice visually.
- **Insight:** the raw variables have many high-end outliers, and capping bounds them without removing rows, which protects the model from extreme values while retaining all clients.

Figure 7: Bivariate analysis



- **What it shows:** relationships between pairs of variables and the target.
- **Why created:** the bivariate analysis criterion (CO4).
- **Insight:** [confirm: defaulters' median LIMIT_BAL is lower/higher than non-defaulters'; which age group has the highest rate; whether the scatter shows an overlap between classes]. Describe only what your figure shows.

Figure 8: Correlation heatmap



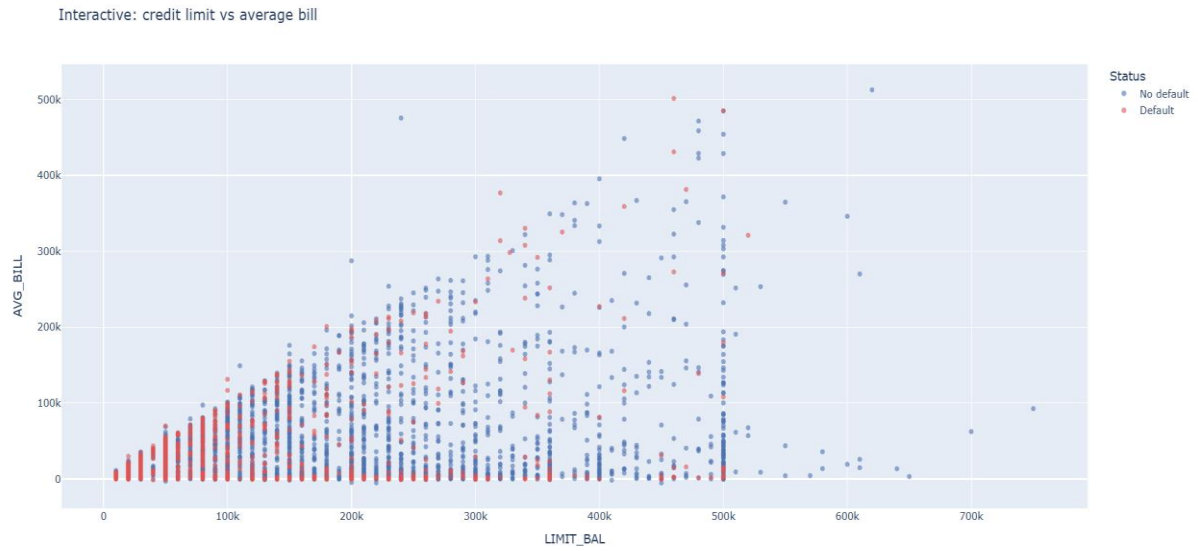
- **What it shows:** pairwise linear correlation among numeric features, and each feature's correlation with the target.
- **Why created:** to detect multicollinearity and rank predictors.
- **Insight:** the BILL_AMT columns are strongly intercorrelated (minimum [confirm]), which justifies the aggregate features. The features most correlated with default are [confirm: expected PAY_x / AVG_DELAY], with correlations of about [confirm]. All correlations with the target are modest, so no single linear relationship is strong.

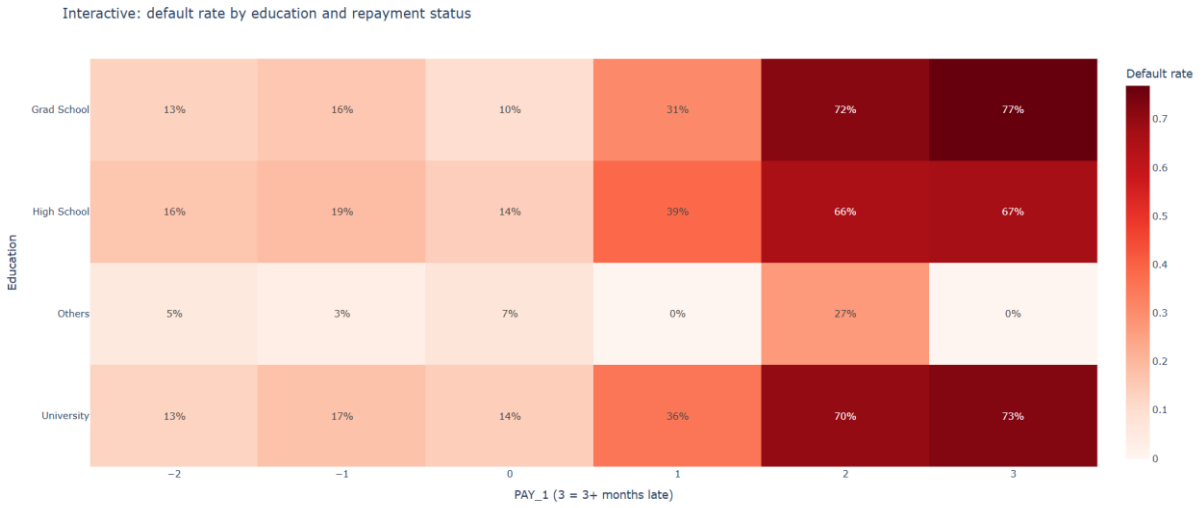
Figure 9: Pairplot



- **What it shows:** scatterplots and marginal densities for five key features, by class.
- **Why created:** to see which features separate the classes and whether relationships are non-linear.
- **Insight:** AVG_DELAY [confirm: separates the classes most visibly], while the other features overlap heavily. The overlap suggests a ceiling on achievable accuracy.

Figures 10-11: Interactive visuals





- **What they show:** zoomable, hoverable versions of the data. The heatmap shows the default rate across education level and repayment status.
- **Why created:** the Interact stage. The HTML files are in the GitHub figures/ folder.
- **Insight:** within every education group the default rate climbs as repayment delay increases, so repayment status explains more than education.

7. Deep Learning Model

Because the data are tabular and numeric after encoding, a Multilayer Perceptron (MLP) was chosen. CNNs and RNNs suit images and sequences and would not be appropriate here.

TABLE 5: Architecture and training settings

Item	Value
Architecture	Input(30) → Dense(64, ReLU) + Dropout(0.2) → Dense(32, ReLU) + Dropout(0.2) → Dense(1, Sigmoid)
Parameters	4,097 [confirm: Cell 18]
Loss	Binary cross-entropy
Optimizer	Adam, lr = 0.001, halved on plateau (min 1e-5)
Regularization	L2 (1e-4), Dropout 0.2, EarlyStopping (patience 10, monitor val AUC)
Batch size / max epochs	256 / 100
Epochs trained	48 (best epoch 38, restored)
Class weights	Balanced, computed from the training set [confirm: values from Cell 12]

Feature analysis:

Before modelling, features were ranked using mutual information and random forest importance. After training, permutation importance on the validation set measured how much AUC fell when each feature was shuffled.

Figure 12: Top 15 features by mutual information and random forest importance.

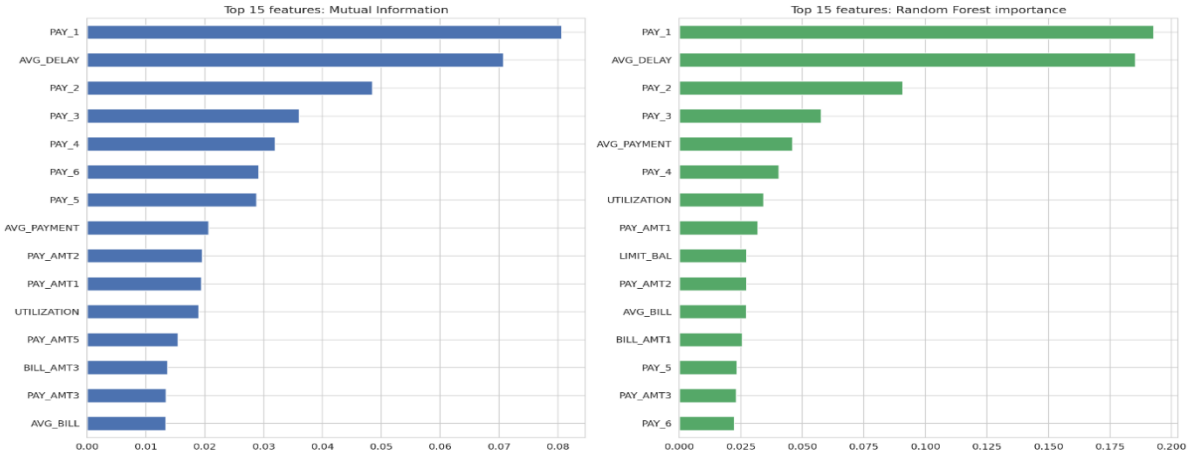


Figure 13: MLP permutation importance (top 15).

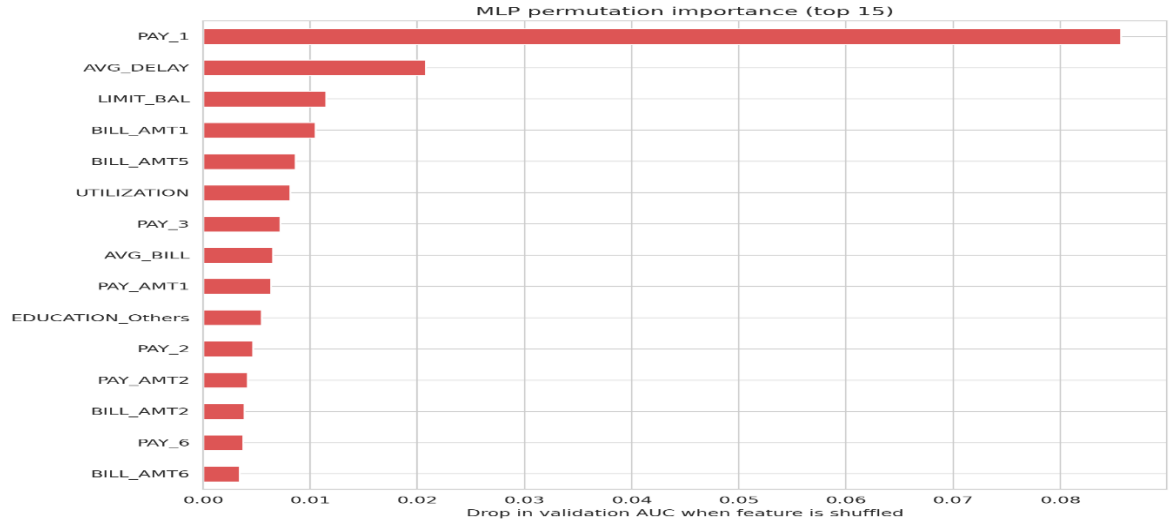


TABLE 6: Top permutation importances

Feature	AUC drop
PAY_1	0.0857
AVG_DELAY	0.0208
LIMIT_BAL	0.0115
BILL_AMT1	0.0105
BILL_AMT5	0.0087

Interpretation:

PAY_1 dominates: shuffling it reduces validation AUC by 0.086, over four times that of the next feature. This agrees with Figure 2, where default rate rose steeply with delay, and with Figures 8 and 9. BILL_AMT5 appearing in the top five is probably an effect of multicollinearity among bill columns, because shuffling one column of a correlated group changes the information the model sees, so individual bill importances should not be over-interpreted.

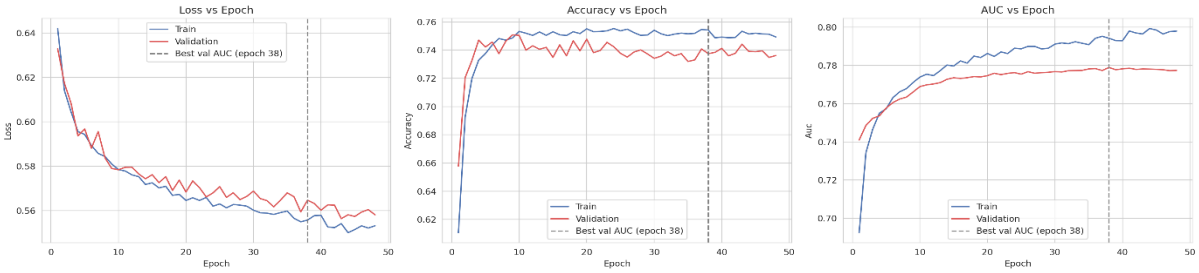
Hyperparameter tuning:

A grid search over hidden layer sizes ((32,16), (64,32), (128,64)), dropout (0.2, 0.4) and learning rate (0.001, 0.003) trained 12 configurations, each with early stopping, scored on **validation** AUC so the test set stayed untouched. The best was **(64, 32), dropout 0.2, lr 0.001, with validation AUC 0.7789**.

Table 7: Hyperparameter search results

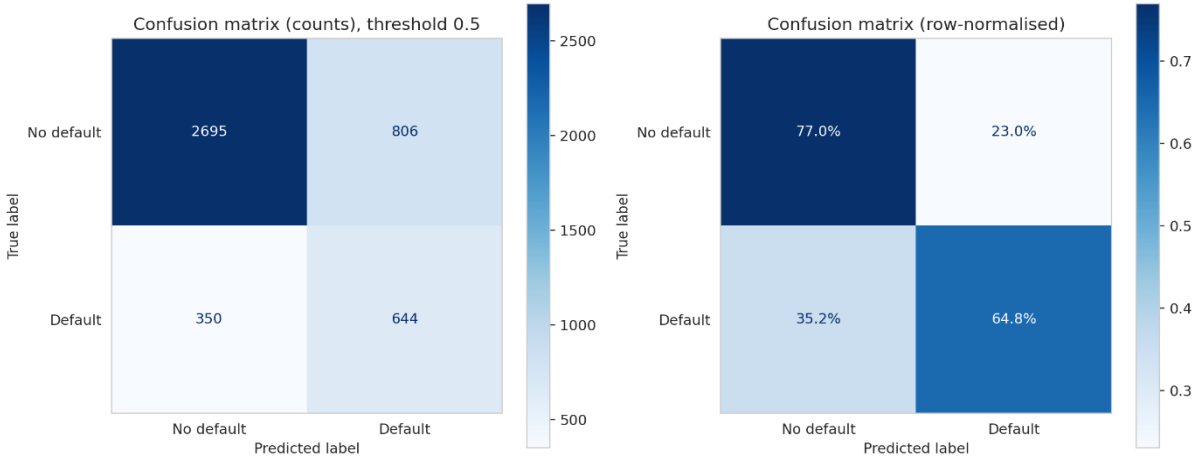
units	dropout	lr	best_val_auc	epochs	time_s
(64, 32)	0.2	0.001	0.7789	40	53.1
(64, 32)	0.2	0.003	0.7776	19	21.1
(128, 64)	0.4	0.001	0.7773	27	17.6
(128, 64)	0.2	0.001	0.7772	21	14.4
(64, 32)	0.4	0.003	0.7771	23	12.7
(64, 32)	0.4	0.001	0.7768	40	20.9
(32, 16)	0.4	0.003	0.7761	32	15.2
(128, 64)	0.4	0.003	0.776	15	11.3
(32, 16)	0.2	0.001	0.7756	40	41.6
(32, 16)	0.2	0.003	0.7751	19	15.3
(128, 64)	0.2	0.003	0.7742	15	11.2
(32, 16)	0.4	0.001	0.772	40	20.4

Figure 14: Loss, accuracy and AUC versus epoch.



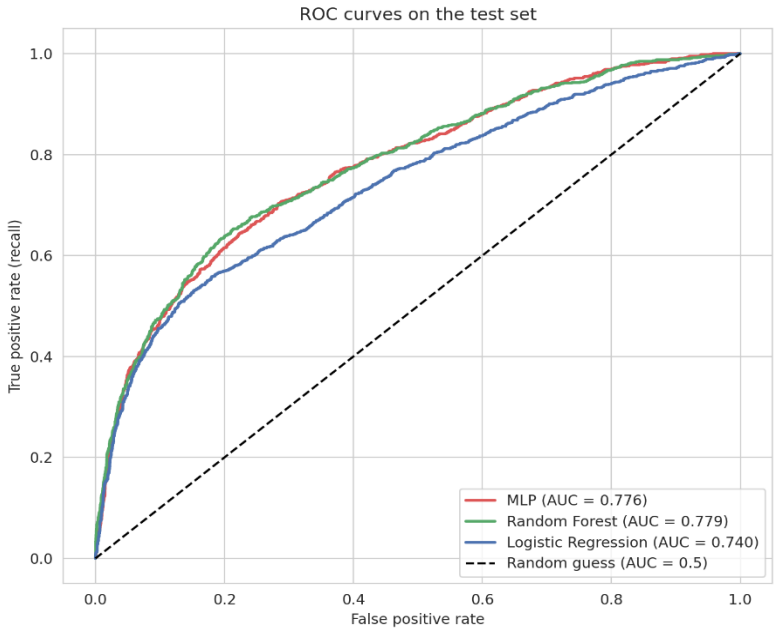
Training and validation curves are shown in Figure 14. Training ran 48 epochs, and early stopping restored the weights from epoch 38, where validation AUC peaked (0.7789). This indicates [limited overfitting, helped by dropout and L2 regularization]. Accuracy is computed at a 0.5 cutoff, and class weighting pushes predictions toward the minority class, so accuracy appears lower than it would for a majority-class model.

Figure 15: Confusion matrix on the test set (threshold 0.5), counts and row-normalised.



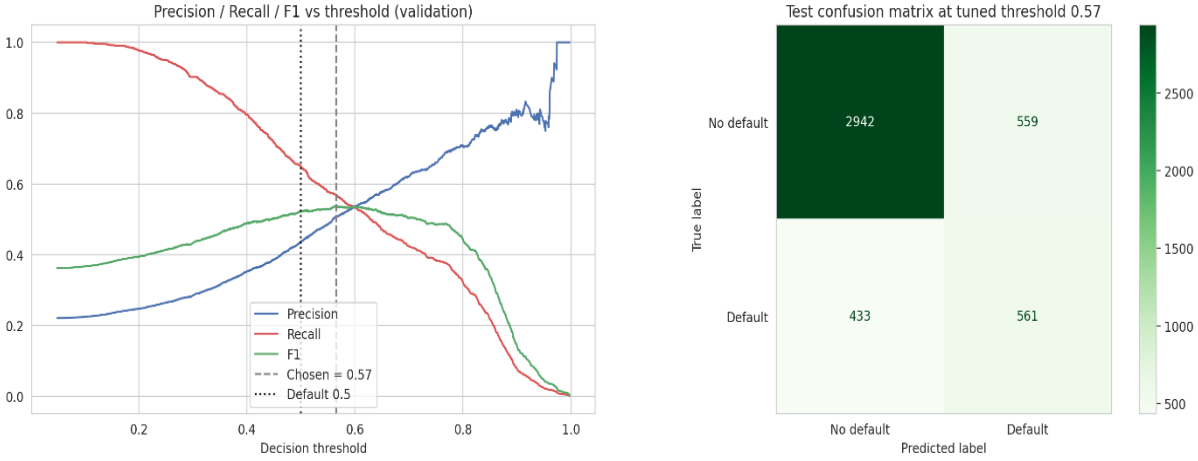
At threshold 0.5 the MLP detected **64.8%** of defaulters (recall 0.648) with precision 0.444, so slightly more than half of the clients it flagged were false alarms.

Figure 16: ROC curves of the MLP, random forest and logistic regression on the test set.



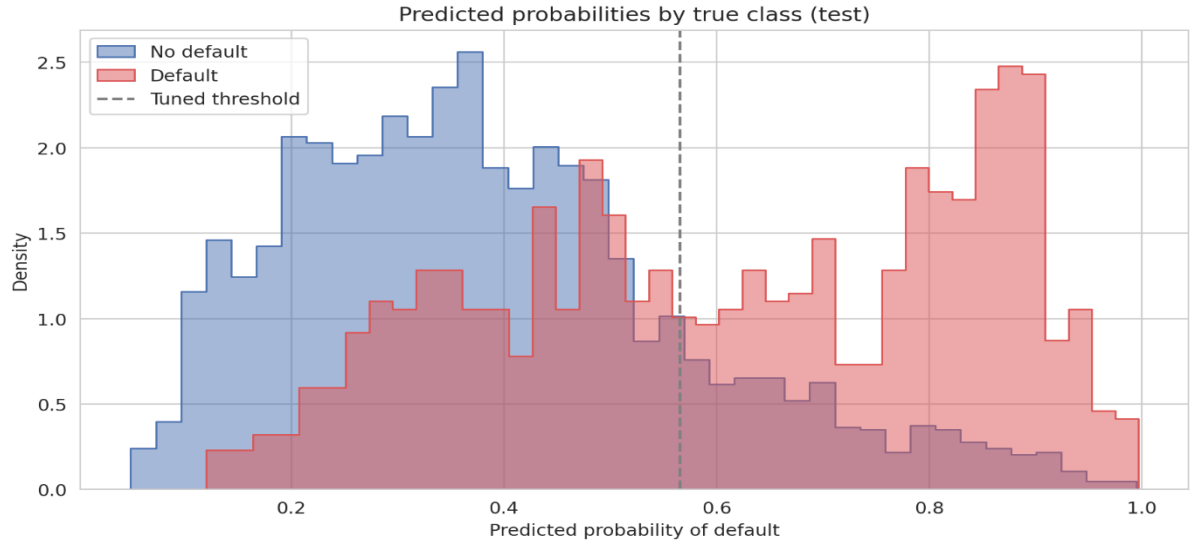
The MLP reached a test ROC AUC of **0.776 (95% bootstrap CI 0.759-0.792)**, consistent with its validation AUC of 0.779, so there is no sign that the tuning overfit the validation set. Logistic regression reached 0.740, below the MLP's confidence interval, which suggests the MLP captures non-linear structure that a linear model misses. Random forest reached 0.779, which lies inside the MLP's interval, so the difference between them is not meaningful. The two curves are nearly indistinguishable.

Figure 17: Precision, recall and F1 versus threshold (validation) and the test confusion matrix at the tuned threshold.



The F1-maximizing threshold on the validation set was **0.57**. Raising it from 0.5 increased precision (0.444 → 0.501) and accuracy (0.743 → 0.779) but reduced recall (0.648 → 0.564), while F1 improved only marginally (0.527 → 0.531). The best threshold depends on the lender's costs: a lender prioritizing the avoidance of losses would choose a lower threshold, while one wanting fewer false alarms would choose a higher one.

Figure 18: Predicted default probability by true class (test set).



The two class distributions overlap substantially, which explains why the AUC stays near 0.78 rather than approaching 0.9. Many defaulters receive low predicted probabilities, so the available features do not fully separate the classes.

TABLE 8: Test-set results

Model	Accuracy	Precision	Recall	F1	ROC AUC	PR AUC
MLP (threshold 0.5)	0.7428	0.4441	0.6479	0.5270	0.7760	0.5342
MLP (threshold 0.57)	0.7793	0.5009	0.5644	0.5307	0.7760	0.5342
Logistic Regression	0.6774	0.3690	0.6459	0.4696	0.7396	0.5050
Random Forest	0.7778	0.4979	0.5996	0.5440	0.7793	0.5593

Table Interpretation: Random forest and the MLP are effectively tied on ROC AUC, and random forest is slightly higher on F1 (0.544) and PR AUC (0.559). The MLP gives a higher recall at the default threshold (0.648 versus 0.600). A classifier predicting "no default" for everyone would score about 77.9% accuracy with 0% recall. The MLP's accuracy at threshold 0.57 (77.9%) therefore matches this trivial baseline, which shows why accuracy is a poor metric here and why AUC, recall and F1 are reported.

9. Conclusion and Future Scope

Conclusion

- **What was learned:** The dataset contains 30,000 clients with 23 features and a 22.12% default rate. It had no missing values, but 35 duplicates, undocumented category codes and heavily skewed monetary variables required cleaning.
- **Key patterns:** Repayment status was the strongest signal. Permutation importance for PAY_1 (0.086 AUC drop) was more than four times that of any other feature, matching the rising default rate with delay in Figure 2. Demographics showed only small differences. The bill-amount columns were highly collinear, which led to engineered aggregate features.
- **Model performance:** The MLP (64-32 hidden units, dropout 0.2) reached a test AUC of 0.776 (95% CI 0.759-0.792) with a recall of 0.648 at threshold 0.5. It clearly outperformed logistic regression (0.740) but was statistically indistinguishable from random forest (0.779).
- **Challenges:** class imbalance, which makes accuracy misleading; skewed variables with many legitimate outliers; multicollinearity among bill columns; heavy class overlap, which limits the achievable AUC; and the trade-off between recall and precision.
- **Limitations:** data from a single country and a six-month window in 2005; no income, employment or macroeconomic variables; demographic attributes raise fairness concerns; the IQR capping limits were computed on the full dataset before the split, which is a minor leakage risk; the threshold was tuned for F1 rather than actual monetary cost; and results come from a single train/validation/test split.

Future scope

- Gradient boosting (XGBoost, LightGBM, CatBoost), which usually performs strongly on tabular data
- Cost-sensitive thresholding based on real loss figures
- SMOTE or focal loss as alternatives to class weights
- SHAP for per-client explanations
- A fairness audit across sex, age and education
- Probability calibration (Platt scaling, isotonic regression)
- K-fold cross-validation and larger searches (Keras Tuner, Optuna)
- LSTM models on the six-month payment sequence
- Additional features such as income, employment and credit bureau data

Literature comparison

Yeh and Lien (2009) compared six data mining techniques on this dataset and reported that artificial neural networks were the most effective at estimating the real probability of

default. In the present study, the MLP clearly outperformed logistic regression (test AUC 0.776 versus 0.740), which agrees with their finding that neural networks handle this problem better than simpler models. However, random forest matched the MLP (AUC 0.779), so the results do not show a clear advantage of neural networks over tree ensembles. This is consistent with general observations that deep learning gains little over ensemble methods on small tabular datasets (Goodfellow et al., 2016).

10. References (APA)

1. Chollet, F. (2015). *Keras*. https://keras.io
2. Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters, 27*(8), 861-874.
3. Fry, B. (2007). *Visualizing data: Exploring and explaining data with the Processing environment*. O'Reilly Media.
4. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
5. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825-2830.
6. Tukey, J. W. (1977). *Exploratory data analysis*. Addison-Wesley.
7. Yeh, I.-C. (2009). *Default of credit card clients* [Dataset]. UCI Machine Learning Repository. https://archive.ics.uci.edu/dataset/350/default+of+credit+card+clients
8. Yeh, I.-C., & Lien, C.-H. (2009). The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients. *Expert Systems with Applications, 36*(2), 2473-2480. https://doi.org/10.1016/j.eswa.2007.12.020

11. Appendix (Code)

The complete code is provided in the notebook notebooks/EDA_Credit_Default.ipynb in the GitHub repository: https://github.com/Rithika-Chandrasekaran/credit-card-default-eda-visualization.

Setup and install

```
!pip install ucimlrepo -q
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

sns.set_style("whitegrid")
pd.set_option("display.max_columns", 50)
RANDOM_STATE = 42
```

Acquire the data (Stage 1: Acquire)

```
from ucimlrepo import fetch_ucirepo
data = fetch_ucirepo(id=350)          # UCI: Default of Credit Card Clients
X = data.data.features.copy()
y = data.data.targets.copy()
# Rename columns by position to meaningful names
X.columns = (["LIMIT_BAL", "SEX", "EDUCATION", "MARRIAGE", "AGE"]
             + ["PAY_1", "PAY_2", "PAY_3", "PAY_4", "PAY_5", "PAY_6"]
             + [f"BILL_AMT{i}" for i in range(1, 7)]
             + [f"PAY_AMT{i}" for i in range(1, 7)])
y.columns = ["default"]
df = pd.concat([X, y], axis=1)
print("Shape:", df.shape)
df.head()
```

Data understanding (Stage 2: Parse)

```
print(df.info())
print("\nMissing values per column:\n", df.isnull().sum().sum(), "total")
print("Duplicate rows:", df.duplicated().sum())
print("\nTarget distribution:\n", df["default"].value_counts())
print("Default rate: {:.2%}".format(df["default"].mean()))
df.describe().T
```

Check categorical codes and data-quality problems

```
for col in ["SEX", "EDUCATION", "MARRIAGE"]:
    print(col, df[col].value_counts().sort_index().to_dict())

print("\nPAY_1 values:", sorted(df["PAY_1"].unique()))
print("Negative BILL_AMT1 rows:", (df["BILL_AMT1"] < 0).sum())
```

Cleaning (Stage 3: Filter)

```
df_clean = df.copy()
# 1. Missing values (this dataset has none, but handle generically)
if df_clean.isnull().sum().sum() > 0:
    num_cols = df_clean.select_dtypes("number").columns
    df_clean[num_cols] = df_clean[num_cols].fillna(df_clean[num_cols].median())

# 2. Duplicates
before = len(df_clean)
df_clean = df_clean.drop_duplicates().reset_index(drop=True)
print(f"Removed {before - len(df_clean)} duplicate rows")

# 3. Fix undocumented category codes
df_clean["EDUCATION"] = df_clean["EDUCATION"].replace({0: 4, 5: 4, 6: 4})  # -> 'others'
df_clean["MARRIAGE"]  = df_clean["MARRIAGE"].replace({0: 3})               # -> 'others'
```

```
# 4. Outliers: IQR winsorizing (cap, don't delete) on continuous amount columns
cont_cols = (["LIMIT_BAL", "AGE"]
             + [f"BILL_AMT{i}" for i in range(1, 7)]
             + [f"PAY_AMT{i}" for i in range(1, 7)])

print("\nOutlier counts (IQR rule) before capping:")
for c in cont_cols:
    q1, q3 = df_clean[c].quantile([0.25, 0.75])
    iqr = q3 - q1
    lo, hi = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    n_out = ((df_clean[c] < lo) | (df_clean[c] > hi)).sum()
    print(f"  {c:10s}: {n_out}")

df_capped = df_clean.copy()
for c in cont_cols:
    q1, q3 = df_clean[c].quantile([0.25, 0.75])
    iqr = q3 - q1
    df_capped[c] = df_clean[c].clip(q1 - 1.5 * iqr, q3 + 1.5 * iqr)

print("\nClean shape:", df_clean.shape)
```

Feature engineering and transformation

```
df_model = df_capped.copy()
# Useful engineered features (helps feature-analysis marks)
bill_cols = [f"BILL_AMT{i}" for i in range(1, 7)]
pay_cols  = [f"PAY_AMT{i}" for i in range(1, 7)]
delay_cols = [f"PAY_{i}" for i in range(1, 7)]

df_model["AVG_BILL"]       = df_model[bill_cols].mean(axis=1)
df_model["AVG_PAYMENT"]    = df_model[pay_cols].mean(axis=1)
df_model["UTILIZATION"]    = df_model["AVG_BILL"] / df_model["LIMIT_BAL"]
df_model["AVG_DELAY"]      = df_model[delay_cols].mean(axis=1)

# Encode categoricals (one-hot). PAY_x are ordinal delay codes -> keep numeric.
df_model["SEX"] = df_model["SEX"].map({1: "Male", 2: "Female"})
df_model["EDUCATION"] = df_model["EDUCATION"].map(
    {1: "GradSchool", 2: "University", 3: "HighSchool", 4: "Others"})
df_model["MARRIAGE"] = df_model["MARRIAGE"].map(
    {1: "Married", 2: "Single", 3: "Others"})
df_model = pd.get_dummies(df_model, columns=["SEX", "EDUCATION", "MARRIAGE"],
                          drop_first=True, dtype=int)
print("Model-ready shape:", df_model.shape)
df_model.head()
```

Train, validation and test split, then scaling

```
X_all = df_model.drop(columns="default")
y_all = df_model["default"]

# 70% train / 15% val / 15% test, stratified to keep the default ratio
X_train, X_temp, y_train, y_temp = train_test_split(
    X_all, y_all, test_size=0.30, stratify=y_all, random_state=RANDOM_STATE)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, stratify=y_temp, random_state=RANDOM_STATE)

# Standardize: fit on TRAIN only (prevents data leakage)
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_val_s   = scaler.transform(X_val)
X_test_s  = scaler.transform(X_test)

print("Train:", X_train_s.shape, "| Val:", X_val_s.shape, "| Test:", X_test_s.shape)
print("Default rate -> train {:.3f}, val {:.3f}, test {:.3f}".format(
      y_train.mean(), y_val.mean(), y_test.mean()))
```

Save processed data (for GitHub)

```
import os
os.makedirs("data", exist_ok=True)
df.to_csv("data/credit_default_raw.csv", index=False)
df_clean.to_csv("data/credit_default_clean.csv", index=False)
df_model.to_csv("data/credit_default_model_ready.csv", index=False)
print("Saved to /content/data/")
```

Helpers and labelled copy

```
import os
from scipy import stats
import plotly.express as px

os.makedirs("figures", exist_ok=True)

def save_fig(name):
    """Save every figure to /figures for the report and GitHub, then show it."""
    plt.tight_layout()
    plt.savefig(f"figures/{name}.png", dpi=200, bbox_inches="tight")
    plt.show()

# Human-readable copy of the cleaned (uncapped) data, used only for plotting
lab = df_clean.copy()
lab["SEX"] = lab["SEX"].map({1: "Male", 2: "Female"})
lab["EDUCATION"] = lab["EDUCATION"].map(
```

```
    {1: "Grad School", 2: "University", 3: "High School", 4: "Others"})
lab["MARRIAGE"] = lab["MARRIAGE"].map({1: "Married", 2: "Single", 3: "Others"})
lab["Status"] = lab["default"].map({0: "No default", 1: "Default"})

overall = lab["default"].mean()
palette = {"No default": "#4C72B0", "Default": "#DD5555"}
```

Mine stage (statistical tests)

```
# Descriptive statistics with skewness and kurtosis (CO4: univariate summaries)
cont4 = ["LIMIT_BAL", "AGE", "BILL_AMT1", "PAY_AMT1"]
print(df_clean[cont4].agg(["mean", "median", "std", "skew", "kurt"]).T.round(2))

# Chi-square: is each categorical variable related to default?
print("\nChi-square tests vs default")
for col in ["SEX", "EDUCATION", "MARRIAGE", "PAY_1"]:
    chi2, p, dof, _ = stats.chi2_contingency(pd.crosstab(df_clean[col], df_clean["default"]))
    print(f"  {col:10s} chi2 = {chi2:10.1f}   p = {p:.2e}")

# Mann-Whitney U: do continuous variables differ between defaulters and non-defaulters?
print("\nMann-Whitney U tests vs default")
for col in cont4:
    a = df_clean.loc[df_clean.default == 1, col]
    b = df_clean.loc[df_clean.default == 0, col]
    u, p = stats.mannwhitneyu(a, b)
        print(f"  {col:10s} median(default) = {a.median():>10.0f}   median(no default) =
{b.median():>10.0f}   p = {p:.2e}")
```

Visual 1: Target balance and default rate by category (univariate/bivariate, discrete)

```
fig, ax = plt.subplots(1, 4, figsize=(20, 4.5))

counts = lab["Status"].value_counts()
ax[0].bar(counts.index, counts.values, color=[palette[i] for i in counts.index])
for i, v in enumerate(counts.values):
    ax[0].text(i, v, f"{v/len(lab):.1%}", ha="center", va="bottom")
ax[0].set_title("Class balance of target"); ax[0].set_ylabel("Number of clients")

for a, col in zip(ax[1:], ["SEX", "EDUCATION", "MARRIAGE"]):
    rate = lab.groupby(col)["default"].mean().sort_values()
    sns.barplot(x=rate.index, y=rate.values, ax=a, color="#4C72B0")
    a.axhline(overall, color="red", ls="--", label=f"Overall = {overall:.1%}")
    for i, v in enumerate(rate.values):
        a.text(i, v, f"{v:.1%}", ha="center", va="bottom")
    a.set_title(f"Default rate by {col}"); a.set_ylabel("Default rate"); a.legend()
save_fig("v1_target_and_categorical")
```

Visual 2: PAY_1 repayment status, discrete distribution with default rate

```
tab = lab.groupby("PAY_1")["default"].agg(["count", "mean"])

fig, ax1 = plt.subplots(figsize=(11, 5))
ax1.bar(tab.index, tab["count"], color="#9ecae1")
ax1.set_yscale("log"); ax1.set_xticks(tab.index)
ax1.set_xlabel("PAY_1: -2 = no consumption, -1 = paid in full, 0 = revolving, 1-8 = months of delay")
ax1.set_ylabel("Number of clients (log scale)")

ax2 = ax1.twinx()
ax2.plot(tab.index, tab["mean"], "o-", color="#DD5555", label="Default rate")
ax2.axhline(overall, color="grey", ls="--", label="Overall rate")
ax2.set_ylabel("Default rate"); ax2.grid(False); ax2.legend(loc="upper left")
ax1.set_title("Distribution of last month's repayment status and default rate")
save_fig("v2_pay1_discrete")
```

Visual 3: Discrete probability distribution (observed vs Binomial)

```
delay_cols = [f"PAY_{i}" for i in range(1, 7)]
late = (df_clean[delay_cols] > 0).sum(axis=1)        # months late out of 6

obs = late.value_counts(normalize=True).sort_index().reindex(range(7), fill_value=0)
p_hat = late.mean() / 6
binom = stats.binom.pmf(range(7), 6, p_hat)

x, w = np.arange(7), 0.4
plt.figure(figsize=(9, 5))
plt.bar(x - w/2, obs.values, w, label="Observed", color="#4C72B0")
plt.bar(x + w/2, binom, w, label=f"Binomial(n=6, p={p_hat:.2f})", color="#F0A33A")
plt.xlabel("Number of late months (out of 6)"); plt.ylabel("Proportion of clients")
plt.title("Late-payment months: observed vs Binomial fit"); plt.legend()
save_fig("v3_binomial_fit")

print(f"Observed mean = {late.mean():.2f}, variance = {late.var():.2f}")
print(f"Binomial variance = {6*p_hat*(1-p_hat):.2f}")
```

Visual 4: Continuous distributions (histogram + KDE)

```
fig, axes = plt.subplots(2, 2, figsize=(13, 9))
for a, c in zip(axes.ravel(), cont4):
    sns.histplot(df_clean[c], kde=True, bins=50, ax=a, color="#4C72B0")
    a.axvline(df_clean[c].mean(), color="red", ls="--", label="Mean")
    a.axvline(df_clean[c].median(), color="green", ls="-", label="Median")
    a.set_title(f"{c}   (skewness = {df_clean[c].skew():.2f})")
    a.set_xlabel(c); a.legend()
save_fig("v4_continuous_distributions")
```

Visual 5: Q-Q plot before and after log transform (Refine stage)

```
raw = df_clean["PAY_AMT1"]
logged = np.log1p(raw)
fig, ax = plt.subplots(1, 4, figsize=(20, 4.5))
sns.histplot(raw, bins=50, ax=ax[0], color="#DD5555");      ax[0].set_title("PAY_AMT1 (raw)")
sns.histplot(logged, bins=50, ax=ax[1], color="#55A868");  ax[1].set_title("PAY_AMT1 (log1p)")
stats.probplot(raw, dist="norm", plot=ax[2]);    ax[2].set_title("Q-Q plot: raw")
stats.probplot(logged, dist="norm", plot=ax[3]); ax[3].set_title("Q-Q plot: log1p")
save_fig("v5_log_transform_qq")
print(f"Skewness raw = {raw.skew():.2f}  ->  log1p = {logged.skew():.2f}")
```

Visual 6: Outliers before and after capping (Filter + Refine)

```
cols = ["LIMIT_BAL", "BILL_AMT1", "PAY_AMT1"]
fig, axes = plt.subplots(2, 3, figsize=(15, 8))
for j, c in enumerate(cols):
    sns.boxplot(y=df_clean[c], ax=axes[0, j], color="#DD5555")
    axes[0, j].set_title(f"{c} before capping")
    sns.boxplot(y=df_capped[c], ax=axes[1, j], color="#55A868")
    axes[1, j].set_title(f"{c} after IQR capping")
save_fig("v6_outliers_before_after")
```

Visual 7: Bivariate analysis (continuous vs target, age groups, scatter)

```
lab["AGE_GROUP"] = pd.cut(lab["AGE"], [20, 30, 40, 50, 60, 80], right=False,
                          labels=["20-29", "30-39", "40-49", "50-59", "60+"])
samp = df_model.sample(3000, random_state=RANDOM_STATE)

fig, ax = plt.subplots(1, 3, figsize=(20, 5))

sns.boxplot(data=lab, x="Status", y="LIMIT_BAL", hue="Status", palette=palette,
            legend=False, order=["No default", "Default"], ax=ax[0])
ax[0].set_title("Credit limit by default status")

rate = lab.groupby("AGE_GROUP", observed=True)["default"].mean()
sns.barplot(x=rate.index, y=rate.values, ax=ax[1], color="#4C72B0")
ax[1].axhline(overall, color="red", ls="--", label="Overall")
ax[1].set_title("Default rate by age group"); ax[1].set_ylabel("Default rate"); ax[1].legend()

sns.scatterplot(data=samp, x="LIMIT_BAL", y="AVG_BILL",
                hue=samp["default"].map({0: "No default", 1: "Default"}),
                palette=palette, alpha=0.5, s=18, ax=ax[2])
ax[2].set_title("Credit limit vs average bill (3,000-client sample)")
ax[2].legend(title="")
save_fig("v7_bivariate")
```

Visual 8: Correlation heatmap (multivariate)

```
bill_cols = [f"BILL_AMT{i}" for i in range(1, 7)]
pay_cols = [f"PAY_AMT{i}" for i in range(1, 7)]
num_cols = (["LIMIT_BAL", "AGE"] + delay_cols + bill_cols + pay_cols
            + ["AVG_BILL", "AVG_PAYMENT", "UTILIZATION", "AVG_DELAY",
"default"])

corr = df_model[num_cols].corr()
mask = np.triu(np.ones_like(corr, dtype=bool))

fig, ax = plt.subplots(1, 2, figsize=(22, 9), gridspec_kw={"width_ratios": [2.2, 1]})
sns.heatmap(corr, mask=mask, cmap="coolwarm", center=0, vmin=-1, vmax=1,
        linewidths=0.3, cbar_kws={"shrink": 0.7}, ax=ax[0])
ax[0].set_title("Correlation heatmap (Pearson)")

tgt = corr["default"].drop("default").sort_values()
ax[1].barh(tgt.index, tgt.values, color=np.where(tgt > 0, "#DD5555", "#4C72B0"))
ax[1].set_title("Correlation of each feature with default")
save_fig("v8_correlation")
```

Visual 9: Pairplot (multivariate)

```
key = ["LIMIT_BAL", "AGE", "AVG_BILL", "AVG_PAYMENT", "AVG_DELAY"]
ps = df_model[key + ["default"]].sample(1500, random_state=RANDOM_STATE).copy()
ps["Status"] = ps["default"].map({0: "No default", 1: "Default"})

g = sns.pairplot(ps.drop(columns="default"), hue="Status", palette=palette,
            corner=True, plot_kws={"alpha": 0.4, "s": 15})
g.fig.suptitle("Pairplot of key engineered features", y=1.02)
g.savefig("figures/v9_pairplot.png", dpi=200, bbox_inches="tight")
plt.show()
```

Visual 10: Interactive visuals (Interact stage)

```
# 10a: Interactive scatter with hover details
s = lab.sample(4000, random_state=RANDOM_STATE).copy()
s["AVG_BILL"] = s[bill_cols].mean(axis=1)
fig = px.scatter(s, x="LIMIT_BAL", y="AVG_BILL", color="Status",
            color_discrete_map=palette, opacity=0.6,
            hover_data=["AGE", "EDUCATION", "PAY_1"],
            title="Interactive: credit limit vs average bill")
fig.write_html("figures/v10a_interactive_scatter.html")
fig.show()

# 10b: Interactive heatmap, default rate by education x repayment status
piv = (lab.assign(PAY_1c=lab["PAY_1"].clip(upper=3))
        .pivot_table(index="EDUCATION", columns="PAY_1c",
```

```
                values="default", aggfunc="mean"))
fig2 = px.imshow(piv, text_auto=".0%", color_continuous_scale="Reds", aspect="auto",
            labels=dict(x="PAY_1 (3 = 3+ months late)", y="Education",
                        color="Default rate"),
            title="Interactive: default rate by education and repayment status")
fig2.write_html("figures/v10b_interactive_heatmap.html")
fig2.show()
```

Download all figures for the report

```
!zip -rq figures.zip figures
from google.colab import files
files.download("figures.zip")
```

Setup

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, regularizers
from sklearn.utils.class_weight import compute_class_weight
from sklearn.metrics import roc_auc_score
import itertools, time, json
keras.utils.set_random_seed(RANDOM_STATE)
feature_names = X_all.columns.tolist()
n_features = X_train_s.shape[1]
ytr, yva, yte = y_train.values, y_val.values, y_test.values
# Class imbalance (~22% defaults): weight the minority class higher
w = compute_class_weight("balanced", classes=np.array([0, 1]), y=ytr)
class_weight = {0: w[0], 1: w[1]}
print("Features:", n_features)
print("Class weights:", {k: round(v, 2) for k, v in class_weight.items()})
```

Feature analysis and feature importance (before modeling)

```
from sklearn.feature_selection import mutual_info_classif
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression

# 1. Mutual information (captures non-linear dependence on the target)
mi = pd.Series(mutual_info_classif(X_train_s, ytr, random_state=RANDOM_STATE),
            index=feature_names).sort_values(ascending=False)

# 2. Random Forest importance (also serves as a baseline model)
rf = RandomForestClassifier(n_estimators=300, max_depth=8, class_weight="balanced",
                    n_jobs=-1, random_state=RANDOM_STATE)
rf.fit(X_train, ytr)
rf_imp                    =                    pd.Series(rf.feature_importances_,
index=feature_names).sort_values(ascending=False)
```

```
fig, ax = plt.subplots(1, 2, figsize=(16, 8))
mi.head(15)[::-1].plot.barh(ax=ax[0], color="#4C72B0")
ax[0].set_title("Top 15 features: Mutual Information")
rf_imp.head(15)[::-1].plot.barh(ax=ax[1], color="#55A868")
ax[1].set_title("Top 15 features: Random Forest importance")
save_fig("v11_feature_importance_pre_model")

# Baselines for comparison with the MLP (validation AUC)
lr = LogisticRegression(max_iter=1000, class_weight="balanced").fit(X_train_s, ytr)
print(f"Baseline Logistic Regression val AUC = {roc_auc_score(yva,
lr.predict_proba(X_val_s)[:,1]):.4f}")
print(f"Baseline Random Forest val AUC = {roc_auc_score(yva,
rf.predict_proba(X_val)[:,1]):.4f}")
```

Model builder

```
def build_mlp(units=(64, 32), dropout=0.3, lr=1e-3, l2=1e-4):
    model = keras.Sequential([keras.Input(shape=(n_features,))])
    for u in units:
        model.add(layers.Dense(u, activation="relu",
                               kernel_regularizer=regularizers.l2(l2)))
        model.add(layers.Dropout(dropout))
    model.add(layers.Dense(1, activation="sigmoid"))
    model.compile(optimizer=keras.optimizers.Adam(learning_rate=lr),
                  loss="binary_crossentropy",
                  metrics=["accuracy", keras.metrics.AUC(name="auc")])
    return model

build_mlp().summary()
```

Basic hyperparameter tuning (grid search on validation AUC)

```
grid = {
    "units":   [(32, 16), (64, 32), (128, 64)],
    "dropout": [0.2, 0.4],
    "lr":      [1e-3, 3e-3],
}

results = []
for units, dropout, lr_ in itertools.product(grid["units"], grid["dropout"], grid["lr"]):
    keras.utils.set_random_seed(RANDOM_STATE)
    m = build_mlp(units, dropout, lr_)
    es = keras.callbacks.EarlyStopping(monitor="val_auc", mode="max", patience=5,
                                       restore_best_weights=True)
    t0 = time.time()
    h = m.fit(X_train_s, ytr, validation_data=(X_val_s, yva), epochs=40,
              batch_size=256, class_weight=class_weight, callbacks=[es], verbose=0)
```

```
    best = max(h.history["val_auc"])
    results.append({"units": str(units), "dropout": dropout, "lr": lr_,
                    "best_val_auc": round(best, 4), "epochs": len(h.history["val_auc"]),
                    "time_s": round(time.time() - t0, 1)})
    print(f"{str(units):10s} dropout={dropout} lr={lr_:<6} -> val AUC {best:.4f}")

tune_df = pd.DataFrame(results).sort_values("best_val_auc", ascending=False).reset_index(drop=True)
tune_df.to_csv("data/hyperparameter_tuning_results.csv", index=False)
tune_df
```

Train the final model with the best hyperparameters

```
best_cfg = tune_df.iloc[0]
best_units = eval(best_cfg["units"])
print("Best configuration:", dict(best_cfg))

keras.utils.set_random_seed(RANDOM_STATE)
model = build_mlp(best_units, best_cfg["dropout"], best_cfg["lr"])

callbacks = [
    keras.callbacks.EarlyStopping(monitor="val_auc", mode="max", patience=10,
                                  restore_best_weights=True),
    keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=4, min_lr=1e-5),
]

history = model.fit(X_train_s, ytr, validation_data=(X_val_s, yva),
                    epochs=100, batch_size=256, class_weight=class_weight,
                    callbacks=callbacks, verbose=2)

print(f"\nEpochs trained: {len(history.history['loss'])}")
print(f"Best val AUC:   {max(history.history['val_auc']):.4f}")

# Save for Step 5 and for GitHub
os.makedirs("model", exist_ok=True)
model.save("model/mlp_credit_default.keras")
pd.DataFrame(history.history).to_csv("data/training_history.csv", index=False)
```

Feature importance for the trained MLP (permutation importance)

```
# Shuffle one feature at a time on the VALIDATION set; the bigger the AUC drop, the more the model relies on it
base_auc = roc_auc_score(yva, model.predict(X_val_s, verbose=0).ravel())
rng = np.random.default_rng(RANDOM_STATE)
drops = {}
for j, name in enumerate(feature_names):
```

```
    scores = []
    for _ in range(5):
        Xp = X_val_s.copy()
        Xp[:, j] = rng.permutation(Xp[:, j])
        scores.append(roc_auc_score(yva, model.predict(Xp, verbose=0).ravel()))
    drops[name] = base_auc - np.mean(scores)
perm = pd.Series(drops).sort_values(ascending=False)
plt.figure(figsize=(9, 8))
perm.head(15)[::-1].plot.barh(color="#DD5555")
plt.xlabel("Drop in validation AUC when feature is shuffled")
plt.title("MLP permutation importance (top 15)")
save_fig("v12_mlp_permutation_importance")
print(perm.head(10).round(4))
```

Summary table for the report (Section 7)

```
summary = {
    "Architecture": f"Input({n_features}) -> " + " -> ".join(
        f"Dense({u}, ReLU)+Dropout({best_cfg['dropout']})" for u in best_units) + " -> Dense(1,
Sigmoid)",
    "Loss": "Binary cross-entropy",
    "Optimizer": f"Adam (lr={best_cfg['lr']}, reduced on plateau)",
    "Regularization": "L2 (1e-4) + Dropout + EarlyStopping (patience 10)",
    "Batch size": 256,
    "Max epochs": 100,
    "Epochs actually trained": len(history.history["loss"]),
    "Class weights": {k: round(v, 2) for k, v in class_weight.items()},
    "Trainable parameters": int(model.count_params()),
    "Split": "70% train / 15% validation / 15% test (stratified)",
}
for k, v in summary.items():
    print(f"{k:26s}: {v}")
json.dump(summary, open("data/model_summary.json", "w"), indent=2, default=str)
```

Test predictions and imports

```
from sklearn.metrics import (confusion_matrix, ConfusionMatrixDisplay, roc_curve,
    classification_report, precision_recall_curve, f1_score, precision_score,
    recall_score, accuracy_score, average_precision_score)

p_val  = model.predict(X_val_s,  verbose=0).ravel()
p_test = model.predict(X_test_s, verbose=0).ravel()

test_auc = roc_auc_score(yte, p_test)

# 95% bootstrap confidence interval for the test AUC
rng = np.random.default_rng(RANDOM_STATE)
```

```
boots = []
for _ in range(500):
    idx = rng.integers(0, len(yte), len(yte))
    boots.append(roc_auc_score(yte[idx], p_test[idx]))
lo, hi = np.percentile(boots, [2.5, 97.5])
print(f"Test AUC = {test_auc:.4f}  (95% CI {lo:.4f} to {hi:.4f})")
print(f"Val  AUC = {roc_auc_score(yva, p_val):.4f}")
```

Loss, accuracy and AUC vs epoch

```
hist = pd.DataFrame(history.history)
best_ep = int(hist["val_auc"].idxmax()) + 1
ep = np.arange(1, len(hist) + 1)

fig, ax = plt.subplots(1, 3, figsize=(19, 4.8))
for a, (tr, va, title) in zip(ax, [("loss", "val_loss", "Loss vs Epoch"),
                                   ("accuracy", "val_accuracy", "Accuracy vs Epoch"),
                                   ("auc", "val_auc", "AUC vs Epoch")]):
    a.plot(ep, hist[tr], label="Train", color="#4C72B0")
    a.plot(ep, hist[va], label="Validation", color="#DD5555")
    a.axvline(best_ep, color="grey", ls="--", label=f"Best val AUC (epoch {best_ep})")
    a.set_title(title); a.set_xlabel("Epoch"); a.set_ylabel(tr.capitalize()); a.legend()
save_fig("v13_training_curves")
print(f"Best epoch by val AUC: {best_ep} of {len(hist)}")
```

Confusion matrix

```
y_pred_05 = (p_test >= 0.5).astype(int)
labels = ["No default", "Default"]

fig, ax = plt.subplots(1, 2, figsize=(12, 5))
ConfusionMatrixDisplay.from_predictions(yte, y_pred_05, display_labels=labels,
                                        cmap="Blues", values_format="d", ax=ax[0])
ax[0].set_title("Confusion matrix (counts), threshold 0.5"); ax[0].grid(False)
ConfusionMatrixDisplay.from_predictions(yte, y_pred_05, display_labels=labels,
                                        cmap="Blues", normalize="true",
                                        values_format=".1%", ax=ax[1])
ax[1].set_title("Confusion matrix (row-normalised)"); ax[1].grid(False)
save_fig("v14_confusion_matrix")

print(classification_report(yte, y_pred_05, target_names=labels, digits=3))
```

ROC curves and AUC (MLP vs baselines)

```
p_lr = lr.predict_proba(X_test_s)[:, 1]
p_rf = rf.predict_proba(X_test)[:, 1]      # the RF was trained on unscaled features

plt.figure(figsize=(7.5, 6.5))
```

```
for name, p, c in [("MLP", p_test, "#DD5555"), ("Random Forest", p_rf, "#55A868"),
                   ("Logistic Regression", p_lr, "#4C72B0")]:
    fpr, tpr, _ = roc_curve(yte, p)
    plt.plot(fpr, tpr, color=c, lw=2, label=f"{name} (AUC = {roc_auc_score(yte, p):.3f})")
plt.plot([0, 1], [0, 1], "k--", label="Random guess (AUC = 0.5)")
plt.xlabel("False positive rate"); plt.ylabel("True positive rate (recall)")
plt.title("ROC curves on the test set"); plt.legend(loc="lower right")
save_fig("v15_roc_curves")
```

Threshold analysis (threshold chosen on validation)

```
# Pick the F1-maximising threshold on the VALIDATION set only
prec, rec, thr = precision_recall_curve(yva, p_val)
f1 = 2 * prec * rec / (prec + rec + 1e-9)
best_thr = float(thr[np.argmax(f1[:-1])])
print(f"Threshold chosen on validation set: {best_thr:.3f}")

fig, ax = plt.subplots(1, 2, figsize=(15, 5))
ax[0].plot(thr, prec[:-1], label="Precision", color="#4C72B0")
ax[0].plot(thr, rec[:-1],  label="Recall",    color="#DD5555")
ax[0].plot(thr, f1[:-1],   label="F1",        color="#55A868")
ax[0].axvline(best_thr, color="grey", ls="--", label=f"Chosen = {best_thr:.2f}")
ax[0].axvline(0.5, color="black", ls=":", label="Default 0.5")
ax[0].set_xlabel("Decision threshold"); ax[0].set_title("Precision / Recall / F1 vs threshold (validation)")
ax[0].legend()

y_pred_t = (p_test >= best_thr).astype(int)
ConfusionMatrixDisplay.from_predictions(yte, y_pred_t, display_labels=labels,
                                        cmap="Greens", values_format="d", ax=ax[1])
ax[1].set_title(f"Test confusion matrix at tuned threshold {best_thr:.2f}"); ax[1].grid(False)
save_fig("v16_threshold_analysis")
```

Probability distributions and the final results table

```
plt.figure(figsize=(9, 5))
for cls, name, c in [(0, "No default", "#4C72B0"), (1, "Default", "#DD5555")]:
    sns.histplot(p_test[yte == cls], bins=40, stat="density", alpha=0.5,
                 color=c, label=name, element="step")
plt.axvline(best_thr, color="grey", ls="--", label="Tuned threshold")
plt.xlabel("Predicted probability of default"); plt.title("Predicted probabilities by true class (test)")
plt.legend()
save_fig("v17_probability_distribution")

# Final comparison table for Section 8 of the report
def row(name, y_true, y_hat, p):
```

```
    return {"Model": name, "Accuracy": accuracy_score(y_true, y_hat),
            "Precision": precision_score(y_true, y_hat), "Recall": recall_score(y_true, y_hat),
            "F1": f1_score(y_true, y_hat), "ROC AUC": roc_auc_score(y_true, p),
            "PR AUC": average_precision_score(y_true, p)}

table = pd.DataFrame([
    row("MLP (threshold 0.5)", yte, y_pred_05, p_test),
    row(f"MLP (threshold {best_thr:.2f})", yte, y_pred_t, p_test),
    row("Logistic Regression", yte, (p_lr >= 0.5).astype(int), p_lr),
    row("Random Forest", yte, (p_rf >= 0.5).astype(int), p_rf),
]).round(4)
table.to_csv("data/test_results_table.csv", index=False)
table
```

Download everything

```
!zip -rq figures.zip figures
!zip -rq outputs.zip data model figures
from google.colab import files
files.download("outputs.zip")
```

Print every number you need in one place

```
print("=== NUMBERS FOR YOUR REPORT ===")
print("Rows, columns (raw):", df.shape, "| model-ready:", df_model.shape)
print("Duplicates removed:", len(df) - len(df_clean))
print("Default rate: {:.2%}".format(df['default'].mean()))
print("Train/Val/Test:", X_train_s.shape[0], X_val_s.shape[0], X_test_s.shape[0])
print("Best config:", dict(best_cfg))
print("Epochs trained:", len(history.history['loss']), "| best epoch:", best_ep)
print(f"Val AUC {max(history.history['val_auc']):.4f} | Test AUC {test_auc:.4f} (CI {lo:.4f}-
{hi:.4f})")
print("\nTop 5 permutation importance:\n", perm.head(5).round(4))
print("\nResults table:\n", table.to_string(index=False))

# Save dependencies for GitHub
import subprocess
open("requirements.txt", "w").write(
      subprocess.run("pip freeze | grep -iE '^(numpy|pandas|matplotlib|seaborn|scikit-
learn|scipy|plotly|tensorflow|ucimlrepo)=='",
            shell=True, capture_output=True, text=True).stdout)
print(open("requirements.txt").read())
```

Print the remaining numbers

```
print("--- Skewness / mean / median ---")
print(df_clean[cont4].agg(["mean", "median", "skew"]).T.round(2))
print("\n--- PAY_1: count and default rate ---")
```

```
print(df_clean.groupby("PAY_1")["default"].agg(["count", "mean"]).round(3))
print("\n--- Late-months (Binomial) ---")
print(f"mean={late.mean():.2f}  var={late.var():.2f}  binomial  var={6*p_hat*(1-p_hat):.2f}
p_hat={p_hat:.3f}")
print("\n--- PAY_AMT1 skew raw -> log1p ---")
print(round(raw.skew(), 2), "->", round(logged.skew(), 2))
print("\n--- Correlation with default ---")
print(corr["default"].drop("default").sort_values().round(2))
print("\n--- BILL_AMT inter-correlation (min) ---")
print(round(df_clean[bill_cols].corr().min().min(), 2))
print("\n--- Confusion matrices (test) ---")
print("thr 0.5 :\n", confusion_matrix(yte, y_pred_05))
print(f"thr {best_thr:.2f}:\n", confusion_matrix(yte, y_pred_t))
print("\nMLP params:", model.count_params())
print("\nTop MI:", list(mi.head(5).index), "\nTop RF:", list(rf_imp.head(5).index))
print("Val AUC  LR:", round(roc_auc_score(yva, lr.predict_proba(X_val_s)[:, 1]), 4),
    " RF:", round(roc_auc_score(yva, rf.predict_proba(X_val)[:, 1]), 4))
tune_df
```